

IoT Deliverable Two

AC/DCI Group

Donell Bicketi- 166604

Gloria Kendi- 166074

Emmanuel Douglas- 168053

Crystal Kanana- 169820

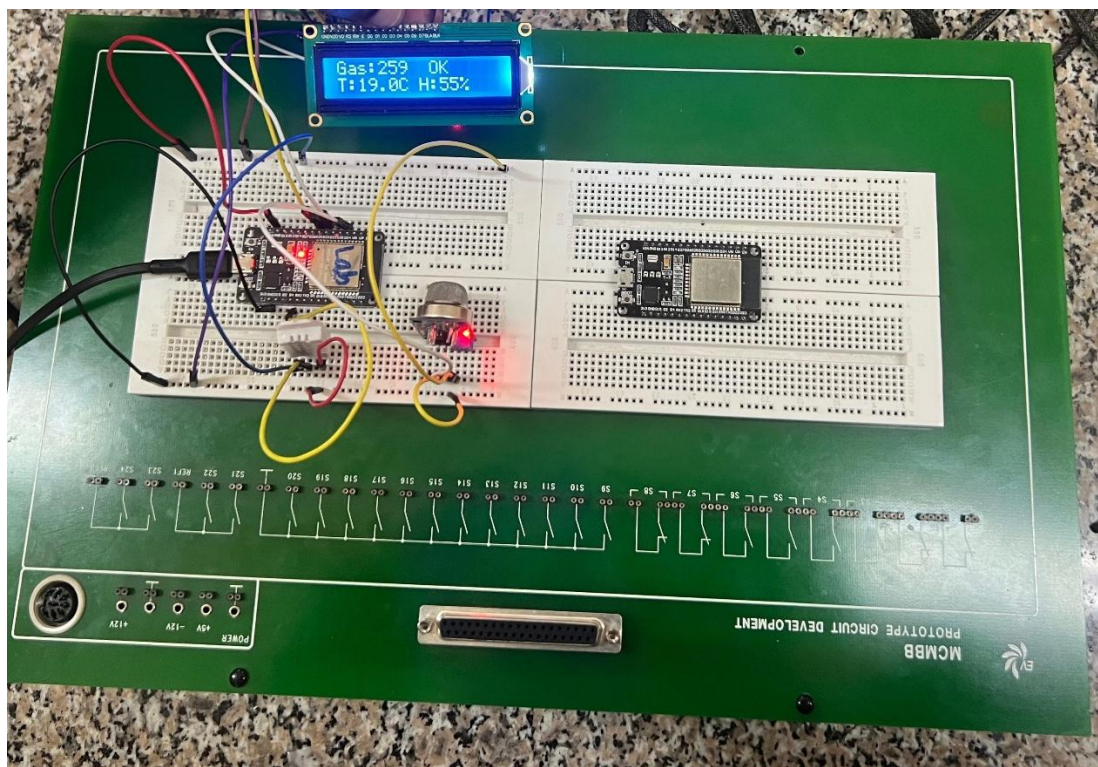
Andrew Kigundu- 166228

1. Architecture A: Single Controller Setup

1 ESP32S connected to 1 MQ-5, 1 DHT22, and 1 LCD (both physical and simulated models)

This architecture combines all environmental monitoring into a single node. The ESP32 reads temperature and humidity from the DHT22 and gas concentration from the MQ-5, displaying the physical environment variables locally on the LCD screen. If parameters fall outside stipulated optimal ranges, it is flagged on the display.

Physical Implementation



Simulated Model

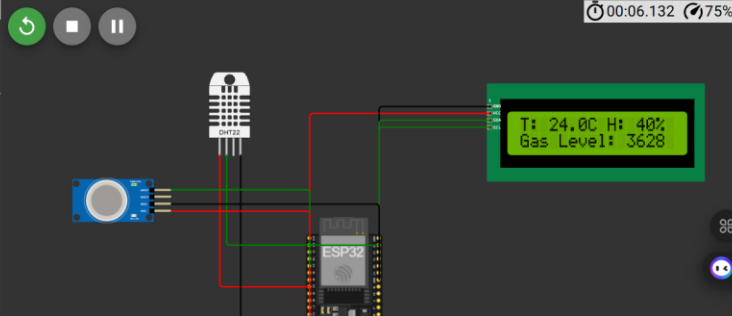
<https://wokwi.com/projects/468456071711542273>

sketch.ino • diagram.json • Library Manager • libraries.txt

```
1 #include <DHT.h>
2 #include <Wire.h>
3 #include <LiquidCrystal_I2C.h>
4
5 #define DHTPIN 23
6 #define DHTTYPE DHT22
7 #define MQ5_ANALOG_PIN 34
8
9 DHT dht(DHTPIN, DHTTYPE);
10 LiquidCrystal_I2C lcd(0x27, 16, 2);
11
12 void setup() {
13   Serial.begin(115200);
14   dht.begin();
15   lcd.init();
16   lcd.backlight();
17
18   pinMode(MQ5_ANALOG_PIN, INPUT);
19 }
20
21 void loop() {
22   float humidity = dht.readHumidity();
23   float temperature = dht.readTemperature();
24   int gasValue = analogRead(MQ5_ANALOG_PIN);
25
26   // Display locally on LCD
27   lcd.setCursor(0, 0);
28   lcd.print("T: " + String(temperature, 1) + "C H: "
```

Simulation

00:06.132 75%

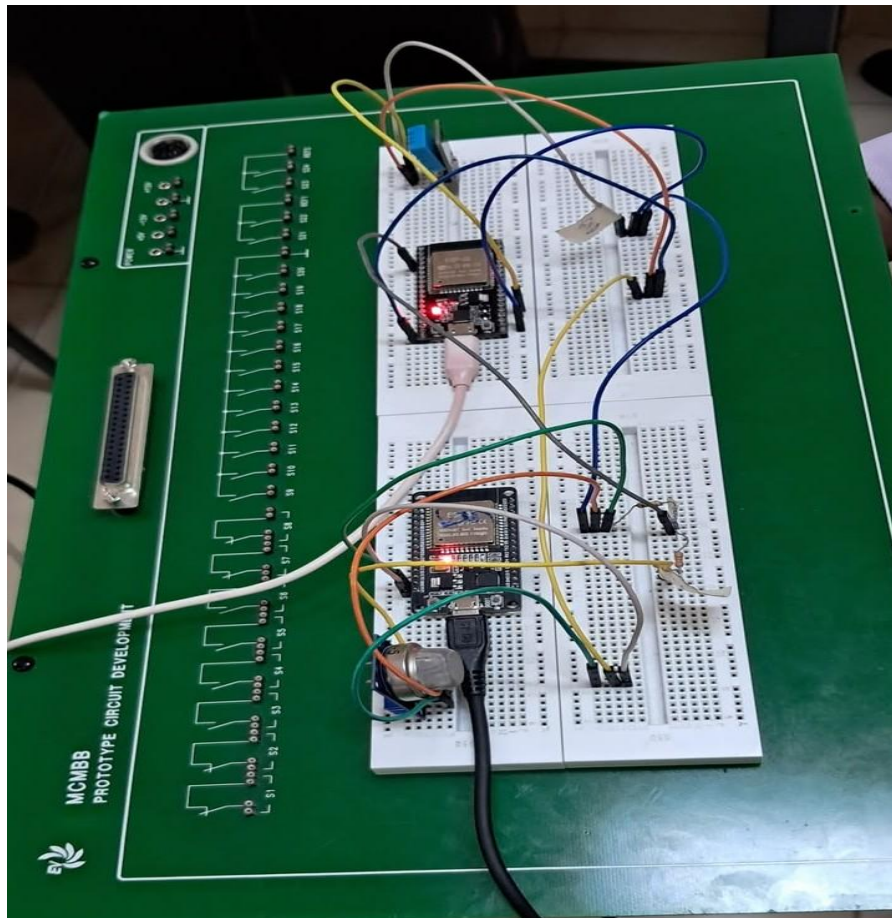


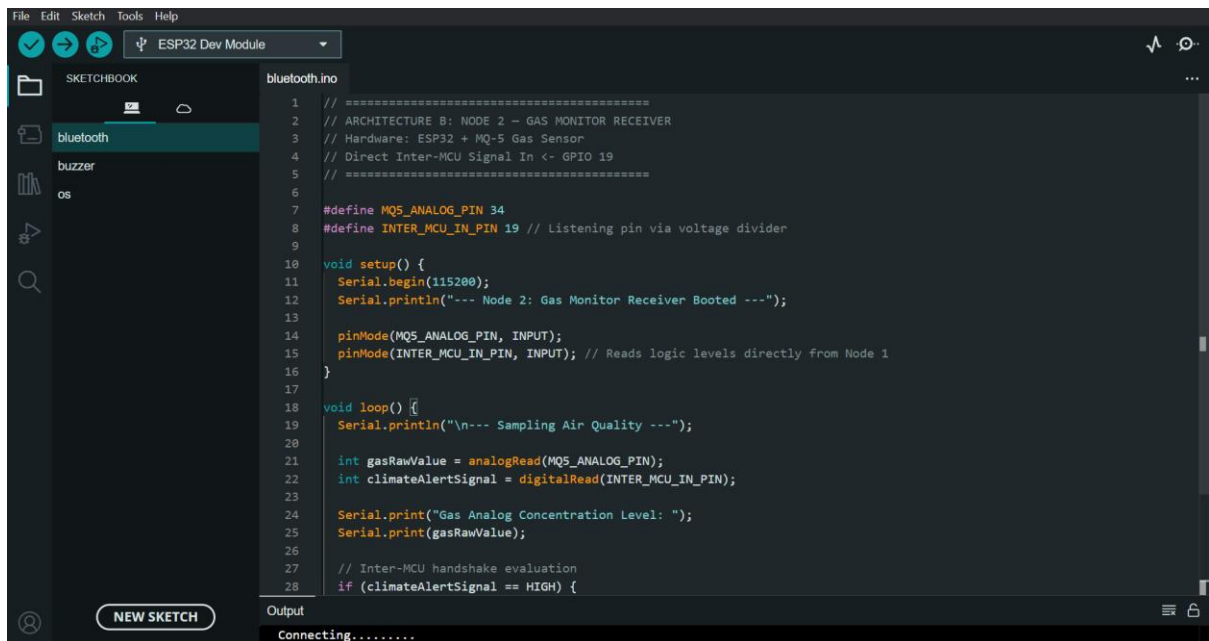
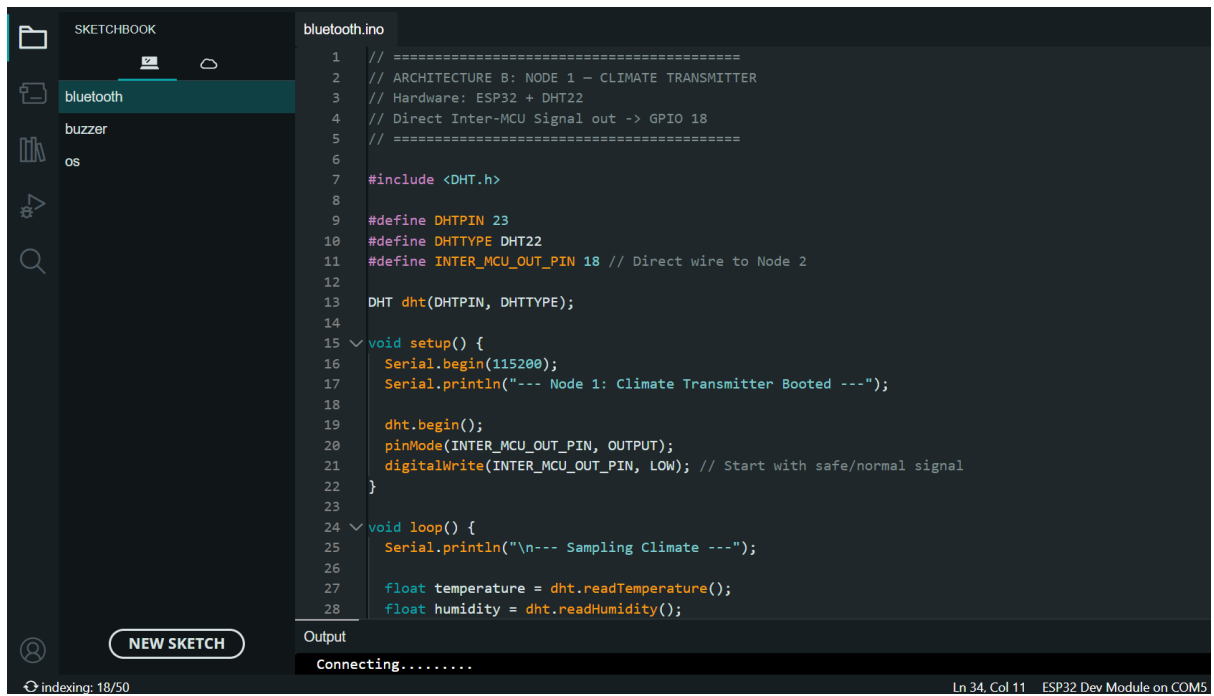
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc

2. Architecture B: Direct Inter-MCU Connection

1 ESP32S connected to 1 MQ-5 interfaced directly with another ESP32S connected to 1 DHT22

In this physical build, the two microcontrollers communicate directly via a shared hardware signaling line without the use of a relay. The first node (Climate Monitor) tracks temperature and humidity. If thresholds are broken, it sends a voltage signal directly from its output pin to the input pin of the second node (Gas Monitor). Node 2 reads this direct hardware line to know if Node 1 is in an alert state, while simultaneously monitoring local gas levels.





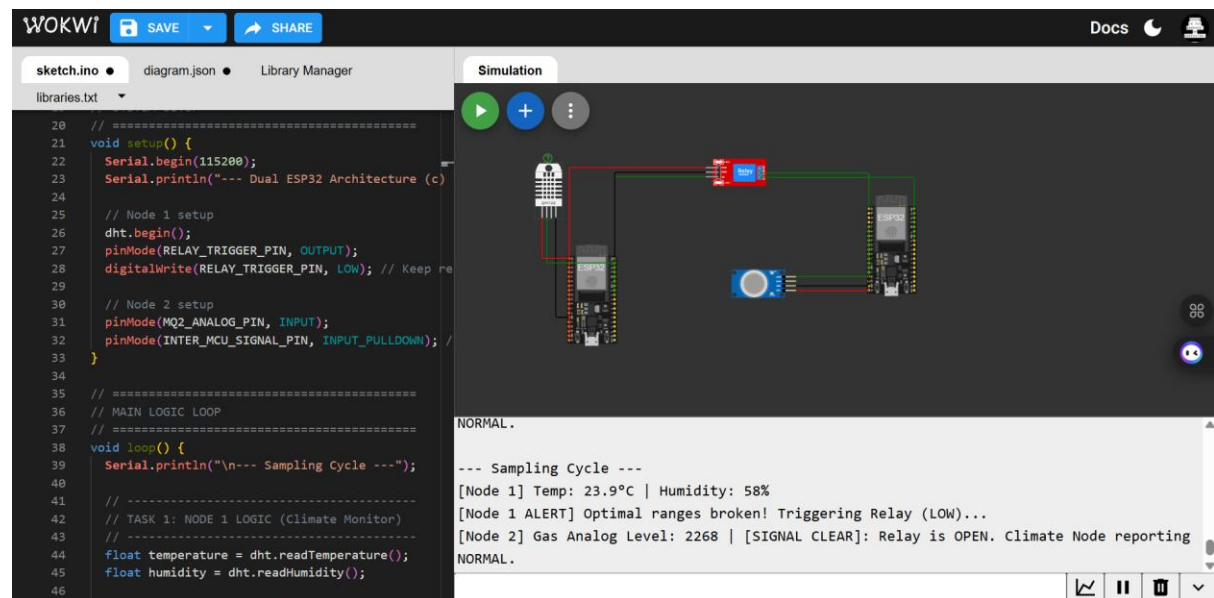
3. Architecture C: Relay-Interfaced Setup

1 ESP32S connected to 1 DHT22 connected to 1 relay which is connected to another ESP32S connected to 1 MQ-5

In this variation, the communication between the two nodes is handled mechanically by a relay. Node 1 triggers the relay when climate ranges are broken, and Node 2 detects the closure of the relay contacts.

Simulated Model:

Original Combined Layout Link: <https://wokwi.com/projects/468455896215550977>



3.1 Simulation Engine Adjustments (The Split-Node Strategy)

When attempting to simulate both ESP32s in a single Wokwi canvas, we encountered a limitation with the virtual serial monitor: Wokwi defaults to displaying the serial output of only one primary microcontroller at a time.

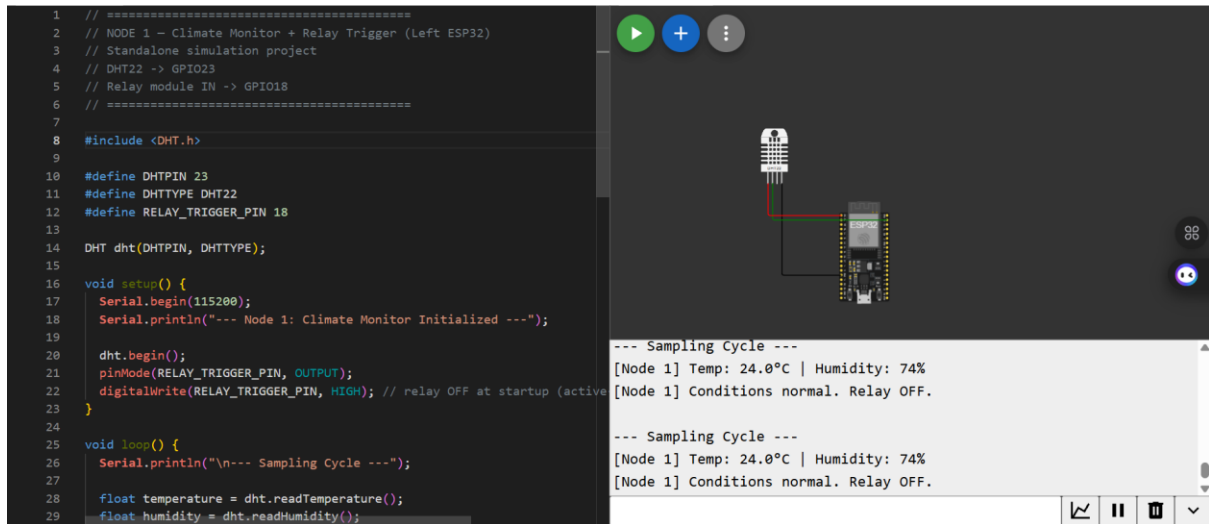
To accurately observe the logic and validate the data outputs of both microcontrollers simultaneously, we separated Architecture C into two standalone simulations:

1. The Climate Node: Simulates the DHT22 and triggers the relay based on temperature/humidity thresholds.
2. The Gas Node: Uses a push-button to simulate the relay's mechanical switch. Pressing the button pulls the ESP32's input pin HIGH, successfully mimicking the alert signal received from the Climate Node's relay.

Split Simulation Links:

Temperature node

<https://wokwi.com/projects/468456246661220353>

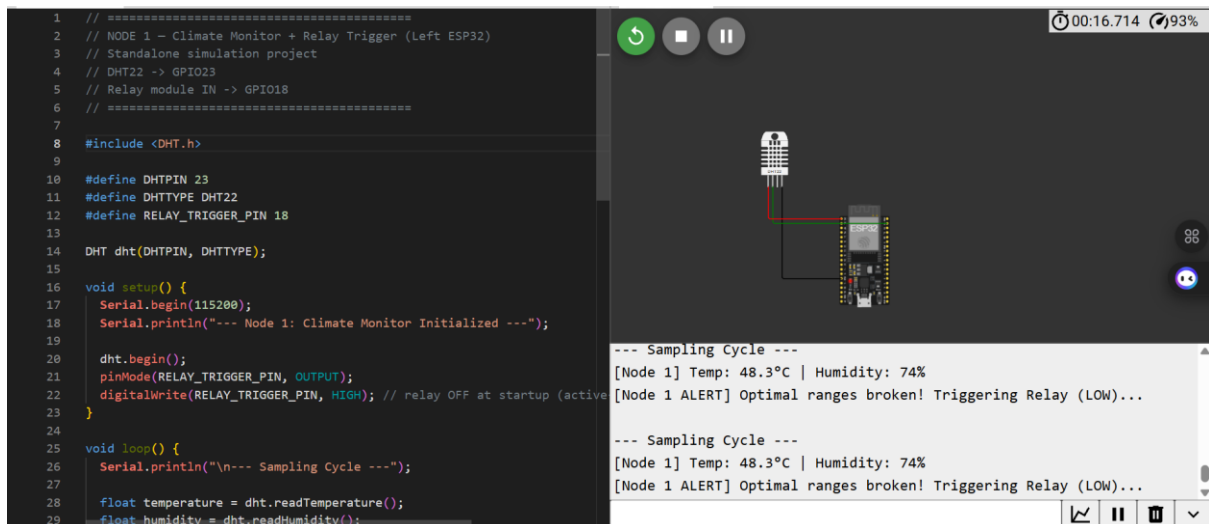


```
1 // =====
2 // NODE 1 - Climate Monitor + Relay Trigger (Left ESP32)
3 // Standalone simulation project
4 // DHT22 -> GPIO23
5 // Relay module IN -> GPIO18
6 // =====
7
8 #include <DHT.h>
9
10 #define DHTPIN 23
11 #define DHTTYPE DHT22
12 #define RELAY_TRIGGER_PIN 18
13
14 DHT dht(DHTPIN, DHTTYPE);
15
16 void setup() {
17   Serial.begin(115200);
18   Serial.println("--- Node 1: Climate Monitor Initialized ---");
19
20   dht.begin();
21   pinMode(RELAY_TRIGGER_PIN, OUTPUT);
22   digitalWrite(RELAY_TRIGGER_PIN, HIGH); // relay OFF at startup (active low)
23 }
24
25 void loop() {
26   Serial.println("\n--- Sampling Cycle ---");
27
28   float temperature = dht.readTemperature();
29   float humidity = dht.readHumidity();
```

--- Sampling Cycle ---
[Node 1] Temp: 24.0°C | Humidity: 74%
[Node 1] Conditions normal. Relay OFF.

--- Sampling Cycle ---
[Node 1] Temp: 24.0°C | Humidity: 74%
[Node 1] Conditions normal. Relay OFF.

Triggered relay not in the optimal range



```
1 // =====
2 // NODE 1 - Climate Monitor + Relay Trigger (Left ESP32)
3 // Standalone simulation project
4 // DHT22 -> GPIO23
5 // Relay module IN -> GPIO18
6 // =====
7
8 #include <DHT.h>
9
10 #define DHTPIN 23
11 #define DHTTYPE DHT22
12 #define RELAY_TRIGGER_PIN 18
13
14 DHT dht(DHTPIN, DHTTYPE);
15
16 void setup() {
17   Serial.begin(115200);
18   Serial.println("--- Node 1: Climate Monitor Initialized ---");
19
20   dht.begin();
21   pinMode(RELAY_TRIGGER_PIN, OUTPUT);
22   digitalWrite(RELAY_TRIGGER_PIN, HIGH); // relay OFF at startup (active low)
23 }
24
25 void loop() {
26   Serial.println("\n--- Sampling Cycle ---");
27
28   float temperature = dht.readTemperature();
29   float humidity = dht.readHumidity();
```

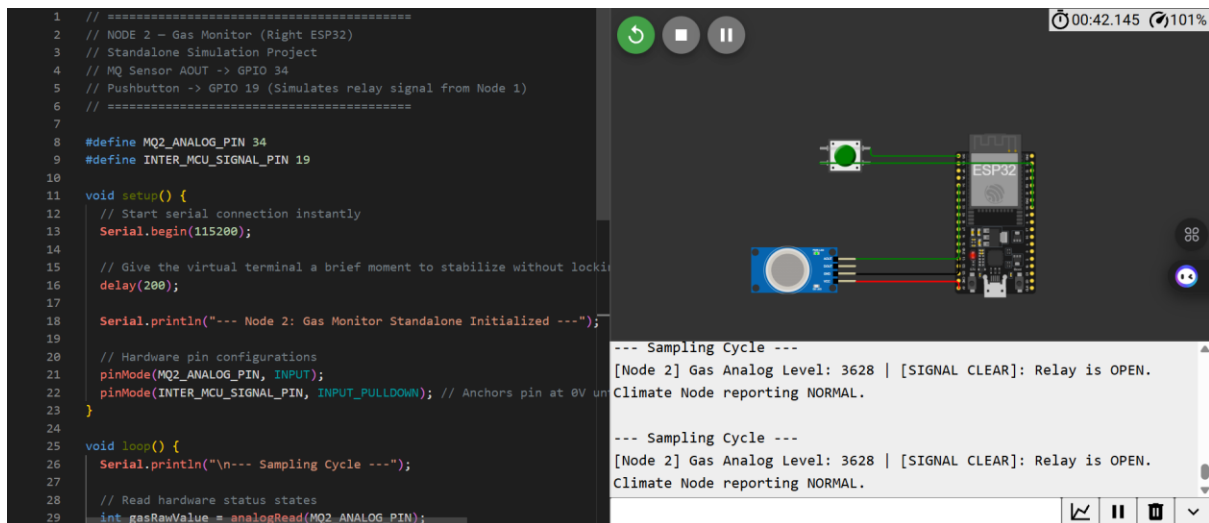
00:16.714 93%

--- Sampling Cycle ---
[Node 1] Temp: 48.3°C | Humidity: 74%
[Node 1 ALERT] Optimal ranges broken! Triggering Relay (LOW)...

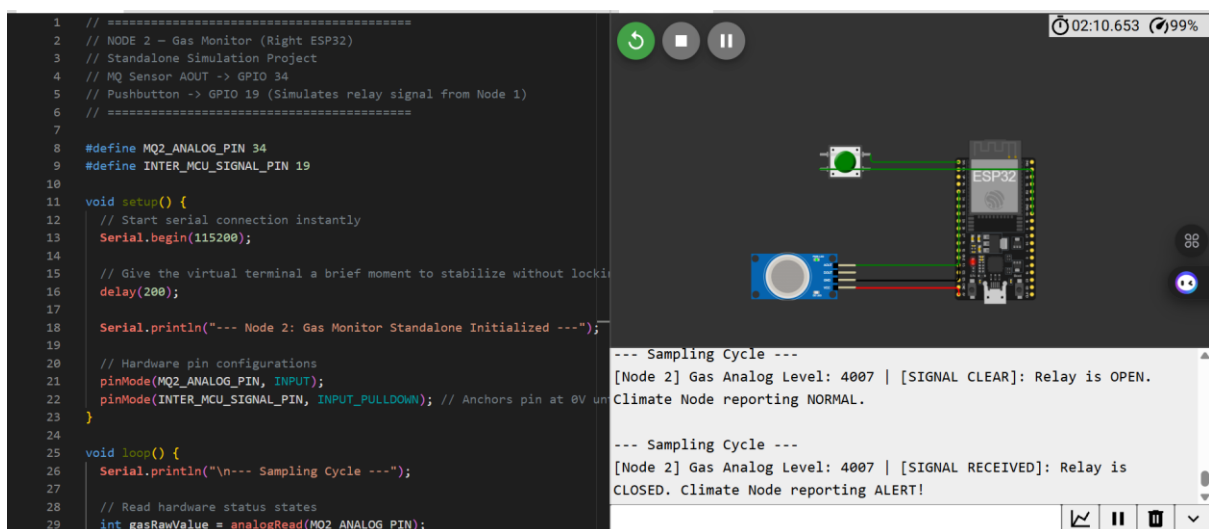
--- Sampling Cycle ---
[Node 1] Temp: 48.3°C | Humidity: 74%
[Node 1 ALERT] Optimal ranges broken! Triggering Relay (LOW)...

Gas node

<https://wokwi.com/projects/468458652421904385>



Relay triggered for gas



4. Physical Implementation Challenges

Transitioning from the Wokwi simulations to physical hardware introduced several real-world debugging challenges that the team had to resolve:

- **Faulty Jumper Wires:** During the dual-node assembly (Architecture B), the inter-MCU communication line kept dropping the signal. We discovered that several breadboard jumper wires had internal micro-breaks. The team had to use a multimeter to run continuity tests on the wires, isolating and discarding the faulty ones to establish a stable connection between the two ESP32 boards.
- **Defective Hardware Components:** Initial tests of Architecture A failed because the physical I2C LCD module was faulty. It refused to initialize properly, displaying solid black blocks or failing to illuminate the backlight entirely. We had to swap the LCD for a

verified working module and run an I2C scanner sketch to confirm the new component's address before the text rendered correctly.

- **Serial Upload Errors:** The ESP32 chips occasionally timed out with an exit status 2 error during code uploads. We resolved this by lowering the upload baud rate in our IDE to 115200 to prevent data corruption caused by electrical noise on the trainer board.

AC/DCI group members

